



TALLER: COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT

YOJANES CHOCUE VELASCO

SEDE

SENA CTPI

Tercer Trimestre

Popayán/Cauca

18/09/2026



TALLER: COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT

YOJANES CHOCUE VELASCO

SEDE

SENA CTPI

Instructor:

Alexandra Muños

Materia

Arquitectura

Tercer Trimestre

Popayán/Cauca

18/09/2026

TALLER: COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT

Programación Orientada a Objetos (POO) – Clases, objetos, métodos y arreglos

Objetivo

Comprender una situación problema, identificar clases, atributos y métodos, y completar código JavaScript utilizando constructor, this, objetos y arreglos.

También se reforzará el uso de push(), pop(), unshift() y shift().

Ejercicio 1 – Biblioteca

Una biblioteca necesita registrar sus libros. Cada libro tiene título, autor y año, y debe poder mostrar su información.

1. ¿Cuál es la clase?

La clase es Libro

2. ¿Cuáles son los atributos?

Los atributos son:

- titulo
- autor
- año

3. ¿Cuál es el método?

El método es mostrarInformacion() porque permite mostrar los datos del libro

4. ¿Qué representa libro1?

libro1 representa un **objeto** creado a partir de la clase Libro.

5. ¿Qué debe escribirse en los espacios?

Código:

```
1 class Libro {
2   constructor(titulo, autor, año) {
3     this.titulo = titulo;
4     this.autor = autor;
5     this.año = año;
6   }
7
8   Qodo: Test this method
9   mostrarInformacion() {
10    console.log("Título: " + this.titulo);
11    console.log("Autor: " + this.autor);
12    console.log("Año: " + this.año);
13  }
14
15  let libro1 = new Libro("Cien años de soledad", "Gabriel García Márquez", 1967);
16
17  libro1.mostrarInformacion();
```

Problems Output Debug Console Terminal Ports Filter (e.g. text, !excludeText, t...) Code

[Running] node "c:\TALLER COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT\COMPRENDER Y COMPLETAR CLASES EN JS\ejercicio.js"

Título: Cien años de soledad
Autor: Gabriel García Márquez
Qodo: Test this method
Año: 1967

[Done] exited with code=0 in 0.742 seconds

Ejercicio 2 – Mascota

La veterinaria registra animales. Cada animal tiene nombre, raza y edad, puede mostrar su información y modificar su edad.

Clase: Mascota

Atributos:

- nombre
- raza
- edad

Métodos:

- mostrarInformacion()
- modificarEdad()

Código:

```
1  class Mascota {
2      constructor(nombre, raza, edad) {
3          this.nombre = nombre;
4          this.raza = raza;
5          this.edad = edad;
6      }
7
8      Qodo: Test this method
9      mostrarInformacion() {
10         console.log("Nombre: " + this.nombre);
11         console.log("Raza: " + this.raza);
12         console.log("Edad: " + this.edad);
13     }
14
15     Qodo: Test this method
16     modificarEdad(nuevaEdad) {
17         this.edad = nuevaEdad;
18     }
19 }
20
21 let mascota1 = new Mascota("Max", "Labrador", 3);
22
23 mascota1.mostrarInformacion();
24
25 mascota1.modificarEdad(4);
26
27 console.log("Nueva edad: " + mascota1.edad);
```

Problems Output Debug Console Terminal Ports Filter (e.g. text, !excludeText, t...) Code

[Running] node "c:\TALLER COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT\COMPRENDER Y COMPLETAR CLASES EN JS\ejercicio mascota.js"

Nombre: Max
Qodo: Test this method
Raza: Labrador
Edad: 3
Nueva edad: 4

[Done] exited with code=0 in 0.789 seconds

Ejercicio 3 – Tienda

La tienda almacena productos con nombre y precio. Debe mostrar el producto y permitir cambiar su precio. También debe mostrar un **antes y un después**.

Clase: Producto

Atributos:

- nombre
- precio

Métodos:

- mostrarProducto()
- cambiarPrecio()

Código:

```
1 class Producto {
2   constructor(nombre, (parameter) precio: any) {
3     this.nombre = nombre;
4     this.precio = precio;
5   }
6
7   Qodo: Test this method
8   mostrarProducto() {
9     console.log("Producto: " + this.nombre);
10    console.log("Precio: $" + this.precio);
11  }
12
13  Qodo: Test this method
14  cambiarPrecio(nuevoPrecio) {
15    this.precio = nuevoPrecio;
16  }
17
18  }
19
20  let producto1 = new Producto("Camiseta", 40000);
21  console.log("Antes:");
22  producto1.mostrarProducto();
23
24  producto1.cambiarPrecio(45000);
25  console.log("Después:");
26  producto1.mostrarProducto();
27
```

Output:

```
[Running] node "c:\TALLER COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT\COMPRENDER Y COMPLETAR CLASES EN JS\ejercicio tienda.js"
Antes:
Producto: Camiseta
Precio: $40000
Después:
Producto: Camiseta
Precio: $45000
[Done] exited with code=0 in 2.467 seconds
```

Antes: Camiseta – \$40.000

Después: Camiseta – \$45.000

Ejercicio 4 – Propietario y mascotas

Un propietario puede tener varias mascotas. Estas deben almacenarse en un arreglo y se deben poder agregar y mostrar.

1. ¿Por qué necesitamos un arreglo?

Porque un propietario puede tener **varias mascotas** y el arreglo permite almacenar todos esos objetos dentro de una misma estructura.

2. ¿Qué método permite agregar una mascota al final?

push().

3. ¿Qué contiene mascota dentro del forEach?

Respuesta: mascota representa cada elemento del arreglo mientras forEach() recorre las mascotas.

Código:

```
1 class Mascota {
2   constructor(nombre, raza, edad) {
3     this.nombre = nombre;
4     this.raza = raza;
5     this.edad = edad;
6   }
7
8   Qodo: Test this method
9   mostrarInformacion() {
10    console.log("Nombre: " + this.nombre);
11    console.log("Raza: " + this.raza);
12    console.log("Edad: " + this.edad);
13  }
14
15  Qodo: Test this method
16  modificarEdad(nuevaEdad) {
17    this.edad = nuevaEdad;
18  }
19 }
20
21 class Propietario {
22   constructor(nombre) {
23     this.nombre = nombre;
24     this.mascotas = [];
25   }
26
27   Qodo: Test this method
28   agregarMascota(mascota) {
29     this.mascotas.push(mascota);
30   }
31
32   Qodo: Test this method
33   mostrarMascotas() {
34     this.mascotas.forEach(mascota => {
35       console.log("Mascota: " + mascota.nombre);
36     });
37   }
38 }
39
40 let propietario1 = new Propietario("Carlos");
41
42 let mascota1 = new Mascota("Max", "Labrador", 3);
43 let mascota2 = new Mascota("Luna", "Poodle", 2);
44
45 propietario1.agregarMascota(mascota1);
46 propietario1.agregarMascota(mascota2);
47
48 propietario1.mostrarMascotas();
```

Problems Output Debug Console Terminal Ports Filter (e.g. text, excludeText, t...) Code

[Running] node "c:\TALLER COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT\COMPRENDER Y COMPLETAR CLASES EN JS\ejercicio 4. propietario y mascotas.js"

Mascota: Max
Mascota: Luna

[Done] exited with code=0 in 0.659 seconds

Ejercicio 5 – Métodos de arreglos

El banco de palabras es push(), pop(), unshift() y shift().

```
45 |
46 let mascotas = ["Max", "Luna", "Michi"];
47
48 // Agregar al final
49 mascotas.push("Rocky");
50
51 // Eliminar del final
52 mascotas.pop();
53
54 // Agregar al inicio
55 mascotas.unshift("Toby");
56
57 // Eliminar del inicio
58 mascotas.shift();
```

Problems Output Debug Console Terminal Ports Filter (e.g. text, excludeText, t...) Code

[Running] node "c:\TALLER COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT\COMPRENDER Y COMPLETAR CLASES EN JS\ejercicio 4. propietario y mascotas.js"
Mascota: Max
Mascota: Luna
[Done] exited with code=0 in 1.325 seconds

Respuestas:

Agregar al final: push()

Eliminar del final: pop()

Agregar al inicio: unshift()

Eliminar del inicio: shift()

Ejercicio 6 – Comprender antes de ejecutar

El ejercicio parte del arreglo ["Max", "Luna", "Michi"] y aplica sucesivamente push(), unshift(), pop() y shift().

Estado inicial:

Max - Luna - Michi

Después de push("Rocky"):

Max - Luna - Michi - Rocky

Después de unshift("Toby"):

Toby - Max - Luna - Michi - Rocky

Después de pop():

Toby - Max - Luna - Michi

Después de shift():

Max - Luna - Michi

Ejercicio 7 – Atención veterinaria

Cada atención veterinaria tiene **fecha, motivo y diagnóstico**.

Clase: Atencion

Atributos:

- fecha
- motivo
- diagnostico

Código:

```
class Atencion {
  constructor(fecha, motivo, diagnostico) {
    this.fecha = fecha;
    this.motivo = motivo;
    this.diagnostico = diagnostico;
  }

  Qodo: Test this method
  mostrarAtencion() {
    console.log("Fecha: " + this.fecha);
    console.log("Motivo: " + this.motivo);
    console.log("Diagnóstico: " + this.diagnostico);
  }
}

let atencion1 = new Atencion(
  "18/09/2026",
  "Vacunación",
  "Animal en buen estado"
);

atencion1.mostrarAtencion();
```

Problems Output Debug Console Terminal Ports Filter (e.g. text, excludeText, t...) Code

[Done] exited with code=0 in 1.325 seconds

[Running] node "c:\TALLER COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT\COMPRENDER Y COMPLETAR CLASES EN JS\ejercicio 6. atencion veterinaria.js"

Fecha: 18/09/2026
Motivo: Vacunación
Diagnóstico: Animal en buen estado

[Done] exited with code=0 in 0.739 seconds

Ejercicio 8 – Integración

Ahora cada animal puede tener **varias atenciones**, por lo que se debe completar un método para registrar una atención.

¿Cómo realizarías esta operación y por qué?

Primero se crea un arreglo dentro de la clase Mascota para almacenar las diferentes atenciones. Después se crea un método llamado registrarAtencion() que utiliza push() para agregar cada nueva atención al arreglo.

¿Qué estructura de datos utilizarías y por qué?

Utilizaría un **arreglo**, porque una mascota puede tener varias atenciones veterinarias.

¿Por qué debe utilizarse una clase?

Porque una clase permite organizar los atributos y métodos relacionados con una mascota y sus atenciones en una misma estructura.

Código:

```
1 class Mascota {
2   constructor(nombre, raza, edad) {
3     this.nombre = nombre;
4     this.raza = raza;
5     this.edad = edad;
6     this.atenciones = [];
7   }
8
9   Qodo: Test this method
10  registrarAtencion(atencion) {
11    this.atenciones.push(atencion);
12  }
13
14  Qodo: Test this method
15  mostrarAtenciones() {
16    this.atenciones.forEach(atencion => {
17      atencion.mostrarAtencion();
18    });
19  }
20 }
```

Problems Output Debug Console Terminal Ports

[Done] exited with code=0 in 1.325 seconds

[Running] node "c:\TALLER COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT\COMPRENDER Y COMPLETAR CLASES EN JS\ejercicio 6. atencion veterinaria.js"

Fecha: 18/09/2026
Motivo: Vacunación
Diagnóstico: Animal en buen estado

[Done] exited with code=0 in 0.739 seconds

Ejercicio final – Estudiante

La institución necesita almacenar las notas de un estudiante. Se debe analizar el arreglo y el método utilizado.

1. ¿Cuál es la clase?

Estudiante.

2. ¿Cuáles son sus atributos?

Los atributos son:

- nombre
- notas

3. ¿Cuál es el método?

agregarNota().

4. ¿Qué tipo de dato debe ser notas?

Debe ser un **arreglo**, porque puede almacenar varias notas.

5. ¿Qué método del arreglo se utiliza?

push().

6. ¿Qué resultado se espera obtener?

Se espera que las notas se agreguen al arreglo del estudiante.

Resultado esperado:

[4.0, 3.5, 5.0]

Código:

```
1 class Estudiante {
2   constructor(nombre) {
3     this.nombre = nombre;
4     this.notas = [];
5   }
6
7   Qodo: Test this method
8   agregarNota(nota) {
9     this.notas.push(nota);
10  }
11 }
12
13 let estudiante1 = new Estudiante("Vojanes");
14
15 estudiante1.agregarNota(4.0);
16 estudiante1.agregarNota(3.5);
17 estudiante1.agregarNota(5.0);
18 console.log(estudiante1.notas);
```

Problems Output Debug Console Terminal Ports

[Done] exited with code=0 in 1.111 seconds

[Running] node "c:\TALLER COMPRENDER Y COMPLETAR CLASES EN JAVASCRIPT\COMPRENDER Y COMPLETAR CLASES EN JS\trabajo final estudiantes.js"

[4, 3.5, 5]

[Done] exited with code=0 in 0.615 seconds

¿Qué método debe utilizar para agregar la nota al inicio del arreglo?

unshift().

Se cambiaría:

this.notas.push(nota);

por:

this.notas.unshift(nota);

Actividad de cierre

El documento pide explicar con palabras propias qué son las clases, objetos, atributos, métodos y arreglos, además de diferenciar los cuatro métodos.

1. ¿Qué es una clase?

Una clase es una estructura que sirve como modelo para crear objetos. En ella se pueden definir atributos y métodos.

2. ¿Qué es un objeto?

Un objeto es una instancia de una clase. Contiene los datos y puede utilizar los métodos definidos en la clase.

3. ¿Qué es un atributo?

Un atributo es una característica o dato que pertenece a un objeto.

Por ejemplo:

nombre
edad
precio

4. ¿Qué es un método?

Un método es una función que pertenece a una clase y permite realizar una acción.

Ejemplo:

mostrarInformacion()

5. ¿Para qué sirve un arreglo dentro de una clase?

Sirve para almacenar varios elementos relacionados con un mismo objeto. Por ejemplo, una mascota puede tener varias atenciones veterinarias.

6. ¿Cuál es la diferencia entre push(), pop(), unshift() y shift()?

- push() agrega un elemento al **final**.
- pop() elimina un elemento del **final**.
- unshift() agrega un elemento al **inicio**.
- shift() elimina un elemento del **inicio**.